

EXPERIMENT 1

Installation of Windows Operating System

Commands Used

winver

systeminfo

hostname

diskpart

list disk

slmgr /xpr

ipconfig

Kali Linux Verification Commands

uname -a

cat /etc/os-release

whoami

hostname

lscpu

df -h

EXPERIMENT 2

Illustrate UNIX Commands and Shell Programming

Shell 1 – Greatest Among Three Numbers

```
#!/bin/bash
```

```
echo "ENTER THREE NUMBERS"
```

```
read a b c
```

```
if [ $a -gt $b ] && [ $a -gt $c ]
```

```
then
```

```
echo "$a is greater"
```

```
elif [ $b -gt $c ]
```

```
then
```

```
echo "$b is greater"
```

```
else
```

```
echo "$c is greater"
```

```
fi
```

Shell 2 – Factorial

```
#!/bin/bash
```

```
echo "ENTER THE NUMBER:"
```

```
read n
fact=1
while [ $n -gt 1 ]
do
fact=$((fact * n))
n=$((n - 1))
done
echo "FACTORIAL OF THE GIVEN NUMBER IS
$fact"
```

Shell 3 – Sum of Odd Numbers up to N

```
#!/bin/bash
echo "ENTER THE RANGE:"
read n
x=1
sum=0
while [ $x -le $n ]
do
sum=$((sum + x))
```

```
x=$((x + 2))
```

```
done
```

```
echo "SUM = $sum"
```

Shell 4 – Fibonacci Numbers

```
#!/bin/bash
```

```
echo "ENTER THE LIMIT:"
```

```
read n
```

```
p=-1
```

```
q=1
```

```
i=1
```

```
while [ $i -le $n ]
```

```
do
```

```
r=$((p + q))
```

```
p=$q
```

CS5302 Operating Systems – Xerox Code | Page

```
2 q=$r
```

```
echo "$r"
```

```
i=$((i + 1))
```

done

Shell 5 – Arithmetic Calculator

```
#!/bin/bash
```

```
echo "ENTER THE VALUE OF A:"
```

```
read a
```

```
echo "ENTER THE VALUE OF B:"
```

```
read b
```

```
echo "ENTER THE OPTION TO PERFORM"
```

```
echo "1. ADDITION"
```

```
echo "2. SUBTRACTION"
```

```
echo "3. MULTIPLICATION"
```

```
echo "4. DIVISION"
```

```
read op
```

```
case "$op" in
```

```
1) echo "Result = $((a + b))" ;;
```

```
2) echo "Result = $((a - b))" ;;
```

```
3) echo "Result = $((a * b))" ;;
```

```
4) echo "Result = $((a / b))" ;;
```

```
*) echo "Invalid Option" ;;
```

```
esac
```

Shell 6 – Largest Digit

```
#!/bin/bash
```

```
echo "ENTER THE NUMBER"
```

```
read a
```

```
max=0
```

```
while [ $a -gt 0 ]
```

```
do
```

```
r=$((a % 10))
```

```
if [ $r -gt $max ]
```

```
then
```

```
max=$r
```

```
fi
```

```
a=$((a / 10))
```

```
done
```

```
echo "THE LARGEST DIGIT OF THE NUMBER:
```

```
$max"
```

Shell 7 – Palindrome String

```
#!/bin/bash
```

```
echo "ENTER THE STRING TO CHECK  
PALINDROME"
```

```
read str
```

```
len=$(echo -n "$str" | wc -c)
```

```
i=1
```

```
j=$((len / 2))
```

```
while [ $i -le $j ]
```

```
do
```

```
k=$(echo "$str" | cut -c $i)
```

```
l=$(echo "$str" | cut -c $len)
```

```
if [ "$k" != "$l" ]
```

```
then
```

```
echo "$str is not a palindrome"
```

```
exit
```

```
fi
```

```
i=$((i + 1))
```

```
len=$((len - 1))
```

```
done
```

```
echo "$str is a palindrome"
```

Shell 8 – Reverse of a Number

```
#!/bin/bash
```

```
echo "ENTER THE NUMBER"
```

```
read n
```

```
rnum=0
```

```
while [ $n -ne 0 ]
```

```
do
```

```
remainder=$((n % 10))
```

```
rnum=$((rnum * 10 + remainder))
```

```
n=$((n / 10))
```

```
done
```

```
echo "REVERSE OF THE NUMBER IS $rnum"
```

EXPERIMENT 3

System Calls: Fork, Exit, Getpid, Wait, Close

3C Program 1 – fork(), getpid(), wait(), exit()

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid < 0)
    {
        printf("Fork Failed\n");
        exit(1);
    }
    else if(pid == 0)
    {
        printf("\nCHILD PROCESS");
```

```
printf("\nChild PID : %d", getpid());
printf("\nParent PID : %d\n", getppid());
exit(0);
}
else
{
wait(NULL);
printf("\nPARENT PROCESS");
printf("\nParent PID : %d", getpid());
printf("\nParent's Parent PID : %d\n", getppid());
}
return 0;
}
```

Shell Script 1

```
#!/bin/bash
echo "Parent Process ID : $$"
(
echo "Child Process ID : $$"
```

```
echo "Parent Process ID : $PPID"
```

```
exit 0
```

```
) &
```

```
wait
```

```
echo "Child Process Completed"
```

C Program 2 – wait()

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
pid_t pid;
```

```
pid = fork();
```

```
if(pid == 0)
```

```
{
```

```
printf("Child Process Running\n");
```

```
sleep(5);
```

```
printf("Child Process Completed\n");
```

```
}  
else  
{  
wait(NULL);  
printf("Parent Resumes Execution\n");  
}  
return 0;  
}
```

Shell Script 2

```
#!/bin/bash  
  
(  
echo "Child Process Running"  
sleep 5  
echo "Child Process Completed"  
) &  
wait  
echo "Parent Resumes Execution"
```

4C Program 3 – close()

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;
    fd = open("sample.txt", O_RDONLY);
    if(fd < 0)
    {
        printf("File Opening Failed\n");
        return 1;
    }
    printf("File Opened Successfully\n");
    close(fd);
    printf("File Closed Successfully\n");
    return 0;
}
```

```
}
```

Shell Script 3

```
#!/bin/bash
```

```
exec 3< sample.txt
```

```
echo "File Opened Successfully"
```

```
exec 3<&-
```

```
echo "File Closed Successfully"
```

EXPERIMENT 4

CPU Scheduling Algorithms

C – FCFS

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int n,i;
```

```
int bt[20],wt[20],tat[20];
```

```
float avg_wt=0,avg_tat=0;
```

```
printf("Enter Number of Processes: ");
```

```
scanf("%d",&n);
```

```
for(i=0;i<n;i++)
{
printf("Enter Burst Time for P%d: ",i+1);
scanf("%d",&bt[i]);
}
wt[0]=0;
for(i=1;i<n;i++)
wt[i]=wt[i-1]+bt[i-1];
for(i=0;i<n;i++)
{
tat[i]=wt[i]+bt[i];
avg_wt+=wt[i];
avg_tat+=tat[i];
}
printf("\nProcess\tBT\tWT\tTAT\n");
for(i=0;i<n;i++)
printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
printf("\nAverage Waiting Time = %.2f",avg_wt/n);
```

```
printf("\nAverage Turnaround Time =  
%.2f\n",avg_tat/n);  
return 0;  
}
```

Shell – FCFS

```
#!/bin/bash  
  
echo "Enter Number of Processes"  
read n  
  
for ((i=0;i<n;i++))  
do  
  
CS5302 Operating Systems – Xerox Code | Page  
5 echo "Enter Burst Time for P$((i+1))"  
read bt[$i]  
done  
  
wt[0]=0  
for ((i=1;i<n;i++))  
do  
  
wt[$i]=$((wt[i-1]+bt[i-1]))
```

```
done
echo
echo -e "Process\tBT\tWT\tTAT"
total_wt=0
total_tat=0
for ((i=0;i<n;i++))
do
tat[$i]=$((wt[i]+bt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))
echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done
avg_wt=$(awk "BEGIN {printf \"%.2f\",
$total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\",
$total_tat/$n}")
echo
echo "Average Waiting Time = $avg_wt"
```

```
echo "Average Turnaround Time = $avg_tat"
```

C – Non-Preemptive SJF

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int n,i,j,temp;
```

```
int bt[20],wt[20],tat[20];
```

```
float avg_wt=0,avg_tat=0;
```

```
printf("Enter Number of Processes: ");
```

```
scanf("%d",&n);
```

```
for(i=0;i<n;i++)
```

```
{
```

```
printf("Enter Burst Time for P%d: ",i+1);
```

```
scanf("%d",&bt[i]);
```

```
}
```

```
for(i=0;i<n-1;i++)
```

```
{
```

```
for(j=i+1;j<n;j++)
```

```
{  
if(bt[i] > bt[j])  
{  
temp=bt[i];  
bt[i]=bt[j];  
bt[j]=temp;  
}  
}  
}  
wt[0]=0;  
for(i=1;i<n;i++)  
wt[i]=wt[i-1]+bt[i-1];  
printf("\nProcess\tBT\tWT\tTAT\n");  
for(i=0;i<n;i++)  
{  
tat[i]=wt[i]+bt[i];  
avg_wt+=wt[i];  
avg_tat+=tat[i];  
}
```

```

printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
}
avg_wt/=n;
avg_tat/=n;
printf("\nAverage Waiting Time = %.2f",avg_wt);
printf("\nAverage Turnaround Time =
%.2f\n",avg_tat);
return 0;
}

```

Shell – SJF

```
#!/bin/bash
```

```
echo "Enter Number of Processes"
```

```
read n
```

CS5302 Operating Systems – Xerox Code | Page

```
6for ((i=0;i<n;i++))
```

```
do
```

```
echo "Enter Burst Time for P$((i+1))"
```

```
read bt[$i]
```

```
done
for ((i=0;i<n-1;i++))
do
for ((j=i+1;j<n;j++))
do
if [ ${bt[i]} -gt ${bt[j]} ]
then
temp=${bt[i]}
bt[$i]=${bt[j]}
bt[$j]=$temp
fi
done
done
wt[0]=0
echo
echo -e "Process\tBT\tWT\tTAT"
total_wt=0
total_tat=0
```

```
for ((i=0;i<n;i++))
do
if [ $i -ne 0 ]
then
wt[$i]=$((wt[i-1]+bt[i-1]))
fi
tat[$i]=$((wt[i]+bt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))
echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done
avg_wt=$(awk "BEGIN {printf \"%.2f\",
$total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\",
$total_tat/$n}")
echo
echo "Average Waiting Time = $avg_wt"
echo "Average Turnaround Time = $avg_tat"
```

C – Priority Scheduling

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int n,i,j,temp;
```

```
int bt[20],pr[20],wt[20],tat[20];
```

```
float avg_wt=0,avg_tat=0;
```

```
printf("Enter Number of Processes: ");
```

```
scanf("%d",&n);
```

```
for(i=0;i<n;i++)
```

```
{
```

```
printf("\nEnter Burst Time for P%d: ",i+1);
```

```
scanf("%d",&bt[i]);
```

```
printf("Enter Priority for P%d: ",i+1);
```

```
scanf("%d",&pr[i]);
```

```
}
```

```
for(i=0;i<n-1;i++)
```

```
{
```

```

for(j=i+1;j<n;j++)
{
if(pr[i] > pr[j])
{
temp=pr[i];
pr[i]=pr[j];
pr[j]=temp;
temp=bt[i];
bt[i]=bt[j];
bt[j]=temp;
}
}
}

wt[0]=0;
for(i=1;i<n;i++)
wt[i]=wt[i-1]+bt[i-1];

```

CS5302 Operating Systems – Xerox Code | Page

7 printf("\nProcess\tPriority\tBT\tWT\tTAT\n");

```
for(i=0;i<n;i++)
{
tat[i]=wt[i]+bt[i];
avg_wt+=wt[i];
avg_tat+=tat[i];
printf("P%d\t%d\t\t%d\t%d\t%d\n",
i+1,pr[i],bt[i],wt[i],tat[i]);
}
avg_wt/=n;
avg_tat/=n;
printf("\nAverage Waiting Time = %.2f",avg_wt);
printf("\nAverage Turnaround Time =
%.2f\n",avg_tat);
return 0;
}
```

Shell – Priority

```
#!/bin/bash
```

```
echo "Enter Number of Processes"
```

```
read n
for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P$((i+1))"
read bt[$i]
echo "Enter Priority for P$((i+1))"
read pr[$i]
done
for ((i=0;i<n-1;i++))
do
for ((j=i+1;j<n;j++))
do
if [ ${pr[i]} -gt ${pr[j]} ]
then
temp=${pr[i]}
pr[$i]=${pr[j]}
pr[$j]=$temp
temp=${bt[i]}
```

```
bt[$i]=${bt[j]}
bt[$j]=$temp
fi
done
done
wt[0]=0
echo
echo -e "Process\tPriority\tBT\tWT\tTAT"
total_wt=0
total_tat=0
for ((i=0;i<n;i++))
do
if [ $i -ne 0 ]
then
wt[$i]=$((wt[i-1]+bt[i-1]))
fi
tat[$i]=$((wt[i]+bt[i]))
total_wt=$((total_wt+wt[i]))
```

```

total_tat=$((total_tat+tat[i]))
echo -e
"P$((i+1))\t${pr[i]}\t\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done
avg_wt=$(awk "BEGIN {printf \"%.2f\",
$total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\",
$total_tat/$n}")
echo
echo "Average Waiting Time = $avg_wt"
echo "Average Turnaround Time = $avg_tat"

```

C – Round Robin

```

#include<stdio.h>

int main()
{
int n,tq,i;
int bt[20],rem_bt[20];
int wt[20]={0},tat[20];

```

```
int time=0,done;
float avg_wt=0,avg_tat=0;
printf("Enter Number of Processes: ");
scanf("%d",&n);
CS5302 Operating Systems – Xerox Code | Page
8
for(i=0;i<n;i++)
{
printf("Enter Burst Time for P%d: ",i+1);
scanf("%d",&bt[i]);
rem_bt[i]=bt[i];
}
```

```
printf("Enter Time Quantum: ");
scanf("%d",&tq);
do
{
done=1;
for(i=0;i<n;i++)
{
```

```
if(rem_bt[i]>0)
{
done=0;
if(rem_bt[i]>tq)
{
time+=tq;
rem_bt[i]-=tq;
}
else
{
time+=rem_bt[i];
wt[i]=time-bt[i];
rem_bt[i]=0;
}
}
}
}while(!done);
printf("\nProcess\tBT\tWT\tTAT\n");
```

```
for(i=0;i<n;i++)
{
tat[i]=bt[i]+wt[i];
avg_wt+=wt[i];
avg_tat+=tat[i];
printf("P%d\t%d\t%d\t%d\n",
i+1,bt[i],wt[i],tat[i]);
}
avg_wt/=n;
avg_tat/=n;
printf("\nAverage Waiting Time = %.2f",avg_wt);
printf("\nAverage Turnaround Time =
%.2f\n",avg_tat);
return 0;
}
```

Shell – Round Robin

```
#!/bin/bash
```

```
echo "Enter Number of Processes"
```

```
read n
for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P$((i+1))"
read bt[$i]
rem_bt[$i]={bt[$i]}
done
echo "Enter Time Quantum"
read tq
time=0
while true
do
done=1
for ((i=0;i<n;i++))
do
if [ ${rem_bt[i]} -gt 0 ]
then
done=0
```

```
if [ ${rem_bt[i]} -gt $tq ]
then
time=$((time+tq))
rem_bt[$i]=$((rem_bt[i]-tq))
else
time=$((time+rem_bt[i]))
wt[$i]=$((time-bt[i]))
rem_bt[$i]=0
```

CS5302 Operating Systems – Xerox Code | Page

9 fi

fi

done

[\$done -eq 1] && break

done

echo

echo -e "Process\tBT\tWT\tTAT"

total_wt=0

total_tat=0

```

for ((i=0;i<n;i++))
do
tat[$i]=$((bt[i]+wt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))
echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done

avg_wt=$(awk "BEGIN {printf \"%.2f\",
$total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\",
$total_tat/$n}")
echo
echo "Average Waiting Time = $avg_wt"
echo "Average Turnaround Time = $avg_tat"

```

EXPERIMENT 5

Inter Process Communication (IPC)

C – IPC Using Pipe

```
#include<stdio.h>
```

```
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<sys/wait.h>
int main()
{
int fd[2];
pid_t pid;
char message[]="Hello from Child Process";
char buffer[100];
pipe(fd);
pid=fork();
if(pid==0)
{
close(fd[0]);
write(fd[1],message,strlen(message)+1);
close(fd[1]);
exit(0);
```

```
}  
else  
{  
wait(NULL);  
close(fd[1]);  
read(fd[0],buffer,sizeof(buffer));  
printf("Message received from child: %s\n",buffer);  
close(fd[0]);  
}  
return 0;  
}
```

Shell – IPC

```
#!/bin/bash  
echo "Hello from Child Process" | cat  
# Alternative Shell Script  
(  
echo "Message from Child Process"  
) | while read msg
```

```
do
echo "Parent Received: $msg"
done
```

EXPERIMENT 6

Semaphore Implementation

C – Mutual Exclusion Using Semaphore

CS5302 Operating Systems – Xerox Code | Page

```
10#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <sys/wait.h>
int main()
{
sem_t *sem;
sem=mmap(NULL,sizeof(sem_t),
PROT_READ|PROT_WRITE,
```

```
MAP_SHARED|MAP_ANONYMOUS,-1,0);
sem_init(&sem,1,1);
if(fork()==0)
{
sem_wait(&sem);
printf("Child Process Entering Critical Section\n");
sleep(3);
printf("Child Process Leaving Critical Section\n");
sem_post(&sem);
exit(0);
}
sem_wait(&sem);
printf("Parent Process Entering Critical
Section\n");
sleep(3);
printf("Parent Process Leaving Critical Section\n");
sem_post(&sem);
wait(NULL);
```

```
sem_destroy(sem);  
return 0;  
}
```

Shell – Lock File

```
#!/bin/bash  
  
LOCKFILE="/tmp/mylock"  
while [ -f "$LOCKFILE" ]  
do  
    sleep 1  
done  
touch "$LOCKFILE"  
echo "Entering Critical Section"  
sleep 5  
echo "Leaving Critical Section"  
rm -f "$LOCKFILE"
```

EXPERIMENT 7

Banker's Algorithm

C

```
#include <stdio.h>

int main()
{
    int n,m,i,j,k;
    int allocation[10][10],max[10][10];
    int need[10][10];
    int available[10];
    int finish[10]={0};
    int safeSeq[10];
    int count=0;
    printf("Enter Number of Processes: ");
    scanf("%d",&n);
    printf("Enter Number of Resources: ");
    scanf("%d",&m);
    printf("\nEnter Allocation Matrix:\n");
    for(i=0;i<n;i++)
    for(j=0;j<m;j++)
    scanf("%d",&allocation[i][j]);
```

```
11 printf("\nEnter Maximum Matrix:\n");
for(i=0;i<n;i++)
for(j=0;j<m;j++)
scanf("%d",&max[i][j]);
printf("\nEnter Available Resources:\n");
for(i=0;i<m;i++)
scanf("%d",&available[i]);
for(i=0;i<n;i++)
for(j=0;j<m;j++)
need[i][j]=max[i][j]-allocation[i][j];
while(count<n)
{
int found=0;
for(i=0;i<n;i++)
{
if(finish[i]==0)
{
```

```
for(j=0;j<m;j++)
if(need[i][j]>available[j])
break;
if(j==m)
{
for(k=0;k<m;k++)
available[k]+=allocation[i][k];
safeSeq[count++]=i;
finish[i]=1;
found=1;
}
}
}
if(found==0)
{
printf("\nSystem is NOT in Safe State\n");
return 0;
}
```

```
}  
printf("\nSystem is in Safe State\n");  
printf("Safe Sequence: ");  
for(i=0;i<n;i++)  
printf("P%d ",safeSeq[i]);  
printf("\n");  
return 0;  
}
```

Shell

```
#!/bin/bash  
echo "Enter Number of Processes:"  
read n  
echo "Enter Safe Sequence (space separated):"  
read -a seq  
echo "Safe Sequence is:"  
for ((i=0;i<n;i++))  
do  
echo -n "P${seq[$i]} "
```

done

echo

echo "System is in Safe State"

EXPERIMENT 8

Deadlock Detection Algorithm

C

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int allocation[10][10],request[10][10];
```

```
int available[10],work[10];
```

```
int finish[10];
```

```
int n,m,i,j,k,found;
```

CS5302 Operating Systems – Xerox Code | Page

```
12 printf("Enter Number of Processes: ");
```

```
scanf("%d",&n);
```

```
printf("Enter Number of Resource Types: ");
```

```
scanf("%d",&m);
```

```
printf("\nEnter Allocation Matrix:\n");
for(i=0;i<n;i++)
for(j=0;j<m;j++)
scanf("%d",&allocation[i][j]);
printf("\nEnter Request Matrix:\n");
for(i=0;i<n;i++)
for(j=0;j<m;j++)
scanf("%d",&request[i][j]);
printf("\nEnter Available Resources:\n");
for(i=0;i<m;i++)
{
scanf("%d",&available[i]);
work[i]=available[i];
}
for(i=0;i<n;i++)
finish[i]=0;
do
{
```

```
found=0;
for(i=0;i<n;i++)
{
    if(finish[i]==0)
    {
        for(j=0;j<m;j++)
        if(request[i][j]>work[j])
            break;
        if(j==m)
        {
            for(k=0;k<m;k++)
            work[k]+=allocation[i][k];
            finish[i]=1;
            found=1;
        }
    }
}
}while(found);
```

```
found=0;
printf("\nDeadlocked Processes:\n");
for(i=0;i<n;i++)
{
if(finish[i]==0)
{
printf("P%d ",i);
found=1;
}
}
if(found==0)
printf("No Deadlock Detected");
printf("\n");
return 0;
}
```

Shell

```
#!/bin/bash
```

```
echo "Enter number of processes:"
```

```
read n
echo "Enter deadlocked process numbers (if
any):"
read processes
if [ -z "$processes" ]
then
echo "No Deadlock Detected"
else
echo "Deadlocked Processes:"
echo "$processes"
CS5302 Operating Systems – Xerox Code | Page
13fi
```

EXPERIMENT 9

Threading

C – POSIX Threads

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
```

```
#include <unistd.h>

void *thread_function(void *arg)
{
    int i;
    for(i=1;i<=5;i++)
    {
        printf("Thread Executing : %d\n",i);
        sleep(1);
    }
    pthread_exit(NULL);
}

int main()
{
    pthread_t t1,t2;
    pthread_create(&t1,NULL,thread_function,NULL);
    pthread_create(&t2,NULL,thread_function,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
}
```

```
printf("All Threads Completed\n");  
return 0;  
}
```

Shell – Concurrent Background Tasks

```
#!/bin/bash
```

```
task1()
```

```
{
```

```
for i in 1 2 3 4 5
```

```
do
```

```
echo "Thread 1 : $i"
```

```
sleep 1
```

```
done
```

```
}
```

```
task2()
```

```
{
```

```
for i in 1 2 3 4 5
```

```
do
```

```
echo "Thread 2 : $i"
```

```
sleep 1
done
}
task1 &
task2 &
wait
echo "All Threads Completed"
```

EXPERIMENT 10

Paging Technique

C

```
#include <stdio.h>
int main()
{
int pageTable[20];
int pageSize,numPages,logicalAddress;
int
pageNumber,offset,frameNumber,physicalAddress;
s;
```

```
int i;
```

CS5302 Operating Systems – Xerox Code | Page

```
14 printf("Enter Page Size: ");
```

```
scanf("%d",&pageSize);
```

```
printf("Enter Number of Pages: ");
```

```
scanf("%d",&numPages);
```

```
printf("Enter Frame Numbers for Each Page:\n");
```

```
for(i=0;i<numPages;i++)
```

```
{
```

```
printf("Page %d -> Frame: ",i);
```

```
scanf("%d",&pageTable[i]);
```

```
}
```

```
printf("Enter Logical Address: ");
```

```
scanf("%d",&logicalAddress);
```

```
pageNumber=logicalAddress/pageSize;
```

```
offset=logicalAddress%pageSize;
```

```
if(pageNumber>=numPages)
```

```
{
```

```
printf("Invalid Logical Address\n");
return 0;
}
frameNumber=pageTable[pageNumber];
physicalAddress=(frameNumber*pageSize)+offset
;
printf("\nPage Number : %d",pageNumber);
printf("\nOffset : %d",offset);
printf("\nFrame Number : %d",frameNumber);
printf("\nPhysical Address :
%d\n",physicalAddress);
return 0;
}
```

Shell

```
#!/bin/bash
```

```
echo "Enter Page Size:"
```

```
read pageSize
```

```
echo "Enter Number of Pages:"
```

```
read numPages
declare -a pageTable
for ((i=0;i<numPages;i++))
do
echo "Enter Frame Number for Page $i:"
read pageTable[$i]
done
echo "Enter Logical Address:"
read logicalAddress
pageNumber=$((logicalAddress / pageSize))
offset=$((logicalAddress % pageSize))
if [ $pageNumber -ge $numPages ]
then
echo "Invalid Logical Address"
exit
fi
frameNumber=${pageTable[$pageNumber]}
```

```
physicalAddress=$((frameNumber * pageSize +  
offset))
```

```
echo "Page Number : $pageNumber"
```

```
echo "Offset : $offset"
```

```
echo "Frame Number : $frameNumber"
```

```
echo "Physical Address : $physicalAddress"
```

EXPERIMENT 11

Memory Allocation Methods – First Fit, Best Fit, Worst Fit

C – First Fit

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
CS5302 Operating Systems – Xerox Code | Page
```

```
15 int blockSize[20],processSize[20];
```

```
int allocation[20];
```

```
int nb,np,i,j;
```

```
printf("Enter Number of Blocks: ");
```

```
scanf("%d",&nb);
printf("Enter Number of Processes: ");
scanf("%d",&np);
printf("Enter Block Sizes:\n");
for(i=0;i<nb;i++)
scanf("%d",&blockSize[i]);
printf("Enter Process Sizes:\n");
for(i=0;i<np;i++)
scanf("%d",&processSize[i]);
for(i=0;i<np;i++)
allocation[i]=-1;
for(i=0;i<np;i++)
{
for(j=0;j<nb;j++)
{
if(blockSize[j]>=processSize[i])
{
allocation[i]=j;
```

```
    blockSize[j]-=processSize[i];  
    break;  
}  
}  
}  
printf("\nProcess No\tProcess Size\tBlock No\n");  
for(i=0;i<np;i++)  
{  
    printf("%d\t\t%d\t\t",i+1,processSize[i]);  
    if(allocation[i]!=-1)  
        printf("%d\n",allocation[i]+1);  
    else  
        printf("Not Allocated\n");  
}  
return 0;  
}
```

Shell – First Fit

```
#!/bin/bash
```

```
echo "First Fit Memory Allocation Demonstration"
blocks=(100 500 200 300 600)
processes=(212 417 112 426)
echo "Memory Blocks: ${blocks[@]}"
echo "Processes: ${processes[@]}"
echo "Allocation Performed Using First Fit"
```

C – Best Fit

```
#include <stdio.h>

int main()
{
    int blockSize[20],processSize[20];
    int allocation[20];
    int nb,np,i,j,bestIdx;
    printf("Enter Number of Blocks: ");
    scanf("%d",&nb);
    printf("Enter Number of Processes: ");
    scanf("%d",&np);
    printf("Enter Block Sizes:\n");
```

```
for(i=0;i<nb;i++)
scanf("%d",&blockSize[i]);
printf("Enter Process Sizes:\n");
for(i=0;i<np;i++)
scanf("%d",&processSize[i]);
for(i=0;i<np;i++)
allocation[i]=-1;
for(i=0;i<np;i++)
```

CS5302 Operating Systems – Xerox Code | Page

```
16 {
bestIdx=-1;
for(j=0;j<nb;j++)
{
if(blockSize[j]>=processSize[i])
{
if(bestIdx==-1 || blockSize[j]<blockSize[bestIdx])
bestIdx=j;
}
}
```

```
}
if(bestIdx!=-1)
{
allocation[i]=bestIdx;
blockSize[bestIdx]-=processSize[i];
}
}
printf("\nProcess No\tProcess Size\tBlock No\n");
for(i=0;i<np;i++)
{
printf("%d\t\t%d\t\t",i+1,processSize[i]);
if(allocation[i]!=-1)
printf("%d\n",allocation[i]+1);
else
printf("Not Allocated\n");
}
return 0;
}
```

Shell – Best Fit

```
#!/bin/bash  
  
echo "Best Fit Memory Allocation Demonstration"  
blocks=(100 500 200 300 600)  
processes=(212 417 112 426)  
echo "Memory Blocks: ${blocks[@]}"  
echo "Processes: ${processes[@]}"  
echo "Allocation Performed Using Best Fit"
```

C – Worst Fit

```
#include <stdio.h>  
  
int main()  
{  
    int blockSize[20],processSize[20];  
    int allocation[20];  
    int nb,np,i,j,worstIdx;  
    printf("Enter Number of Blocks: ");  
    scanf("%d",&nb);  
    printf("Enter Number of Processes: ");
```

```
scanf("%d",&np);
printf("Enter Block Sizes:\n");
for(i=0;i<nb;i++)
scanf("%d",&blockSize[i]);
printf("Enter Process Sizes:\n");
for(i=0;i<np;i++)
scanf("%d",&processSize[i]);
for(i=0;i<np;i++)
allocation[i]=-1;
for(i=0;i<np;i++)
{
worstIdx=-1;
for(j=0;j<nb;j++)
{
if(blockSize[j]>=processSize[i])
{
if(worstIdx==-1 || blockSize[j]>blockSize[worstIdx])
worstIdx=j;
```

```

}
}
if(worstIdx!=-1)
{
allocation[i]=worstIdx;
CS5302 Operating Systems – Xerox Code | Page
17 blockSize[worstIdx]-=processSize[i];
}
}
printf("\nProcess No\tProcess Size\tBlock No\n");
for(i=0;i<np;i++)
{
printf("%d\t\t%d\t\t",i+1,processSize[i]);
if(allocation[i]!=-1)
printf("%d\n",allocation[i]+1);
else
printf("Not Allocated\n");
}

```

```
return 0;
```

```
}
```

Shell – Worst Fit

```
#!/bin/bash
```

```
echo "Worst Fit Memory Allocation  
Demonstration"
```

```
blocks=(100 500 200 300 600)
```

```
processes=(212 417 112 426)
```

```
echo "Memory Blocks: ${blocks[@]}"
```

```
echo "Processes: ${processes[@]}"
```

```
echo "Allocation Performed Using Worst Fit"
```

EXPERIMENT 12

Page Replacement Algorithms

C – FIFO

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int pages[50],frames[10];
```

```
int n,f,i,j,k=0;
int fault=0,found;
printf("Enter Number of Pages: ");
scanf("%d",&n);
printf("Enter Reference String:\n");
for(i=0;i<n;i++)
scanf("%d",&pages[i]);
printf("Enter Number of Frames: ");
scanf("%d",&f);
for(i=0;i<f;i++)
frames[i]=-1;
for(i=0;i<n;i++)
{
found=0;
for(j=0;j<f;j++)
{
if(frames[j]==pages[i])
{
```

```
found=1;
break;
}
}
if(found==0)
{
frames[k]=pages[i];
k=(k+1)%f;
fault++;
}
}
printf("Total Page Faults = %d\n",fault);
return 0;
}
```

Shell – FIFO

```
#!/bin/bash
```

```
echo "FIFO Page Replacement Demonstration"
```

```
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)
```

```
18frames=3
```

```
echo "Reference String: ${pages[@]}"
```

```
echo "Frames: $frames"
```

```
echo "FIFO Algorithm Executed"
```

C – LRU

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int pages[50],frames[10],time[10];
```

```
int n,f,i,j;
```

```
int fault=0,count=0;
```

```
int found,pos,min;
```

```
printf("Enter Number of Pages: ");
```

```
scanf("%d",&n);
```

```
printf("Enter Reference String:\n");
```

```
for(i=0;i<n;i++)
```

```
scanf("%d",&pages[i]);
```

```
printf("Enter Number of Frames: ");
scanf("%d",&f);
for(i=0;i<f;i++)
frames[i]=-1;
for(i=0;i<n;i++)
{
found=0;
for(j=0;j<f;j++)
{
if(frames[j]==pages[i])
{
count++;
time[j]=count;
found=1;
break;
}
}
if(found==0)
```

```
{  
min=time[0];  
pos=0;  
for(j=0;j<f;j++)  
{  
if(frames[j]==-1)  
{  
pos=j;  
break;  
}  
if(time[j]<min)  
{  
min=time[j];  
pos=j;  
}  
}  
frames[pos]=pages[i];  
count++;
```

```
time[pos]=count;
fault++;
}
}
printf("Total Page Faults = %d\n",fault);
return 0;
}
```

Shell – LRU

```
#!/bin/bash
echo "LRU Page Replacement Demonstration"
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)
echo "Reference String: ${pages[@]}"
echo "LRU Algorithm Executed"
CS5302 Operating Systems – Xerox Code | Page
```

19C – Optimal

```
#include <stdio.h>
int main()
{
```

```
int pages[50],frames[10];
int n,f,i,j,k,pos=0;
int fault=0,found;
printf("Enter Number of Pages: ");
scanf("%d",&n);
printf("Enter Reference String:\n");
for(i=0;i<n;i++)
scanf("%d",&pages[i]);
printf("Enter Number of Frames: ");
scanf("%d",&f);
for(i=0;i<f;i++)
frames[i]=-1;
for(i=0;i<n;i++)
{
found=0;
for(j=0;j<f;j++)
{
if(frames[j]==pages[i])
```

```
{  
found=1;  
break;  
}  
}  
if(found==0)  
{  
pos=0;  
for(j=0;j<f;j++)  
{  
int future=999;  
for(k=i+1;k<n;k++)  
{  
if(frames[j]==pages[k])  
{  
future=k;  
break;  
}  
}
```

```
}  
if(j==0 || future>pos)  
pos=future;  
}  
frames[0]=pages[i];  
fault++;  
}  
}  
printf("Total Page Faults = %d\n",fault);  
return 0;  
}
```

Shell – Optimal

```
#!/bin/bash  
  
echo "Optimal Page Replacement Demonstration"  
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)  
echo "Reference String: ${pages[@]}"  
echo "Optimal Algorithm Executed"
```

EXPERIMENT 13

File Organization Techniques

C – Sequential File Organization

```
#include <stdio.h>
```

```
struct student
```

CS5302 Operating Systems – Xerox Code | Page

```
20{
```

```
int regno;
```

```
char name[20];
```

```
};
```

```
int main()
```

```
{
```

```
FILE *fp;
```

```
struct student s;
```

```
fp=fopen("student.dat","w");
```

```
printf("Enter Register Number: ");
```

```
scanf("%d",&s.regno);
```

```
printf("Enter Name: ");
```

```
scanf("%s",s.name);
```

```
fprintf(fp,"%d %s\n",s.regno,s.name);
fclose(fp);
fp=fopen("student.dat","r");
fscanf(fp,"%d %s",&s.regno,s.name);
printf("\nRecord Details\n");
printf("Register Number : %d\n",s.regno);
printf("Name : %s\n",s.name);
fclose(fp);
return 0;
}
```

Shell – Sequential

```
#!/bin/bash
echo "Enter Register Number:"
read regno
echo "Enter Name:"
read name
echo "$regno $name" > student.txt
echo "Contents of File"
```

```
cat student.txt
```

C – Direct (Random) File Organization

```
#include <stdio.h>
```

```
struct student
```

```
{
```

```
int regno;
```

```
char name[20];
```

```
};
```

```
int main()
```

```
{
```

```
FILE *fp;
```

```
struct student s;
```

```
fp=fopen("random.dat","wb+");
```

```
printf("Enter Register Number: ");
```

```
scanf("%d",&s.regno);
```

```
printf("Enter Name: ");
```

```
scanf("%s",s.name);
```

```
fwrite(&s,sizeof(s),1,fp);
```

```
rewind(fp);
fread(&s,sizeof(s),1,fp);
printf("\nRecord Found\n");
printf("Reg No : %d\n",s.regno);
printf("Name : %s\n",s.name);
fclose(fp);
return 0;
}
```

Shell – Direct

```
#!/bin/bash
echo "Enter Record:"
read rec
echo "$rec" > random.txt
echo "Random Access Record"
```

CS5302 Operating Systems – Xerox Code | Page

```
21sed -n '1p' random.txt
```

C – Indexed File Organization

```
#include <stdio.h>
```

```
struct student
{
int regno;
char name[20];
};

int main()
{
struct student s[3];
int key,i;
printf("Enter 3 Student Records\n");
for(i=0;i<3;i++)
scanf("%d %s",&s[i].regno,s[i].name);
printf("Enter Register Number to Search: ");
scanf("%d",&key);
for(i=0;i<3;i++)
{
if(s[i].regno==key)
{
```

```
printf("\nRecord Found\n");
printf("Reg No : %d\n",s[i].regno);
printf("Name : %s\n",s[i].name);
return 0;
}
}
printf("Record Not Found\n");
return 0;
}
```

Shell – Indexed

```
#!/bin/bash
echo "Enter Student Records"
echo "101 Arun" > index.txt
echo "102 Kumar" >> index.txt
echo "103 Ravi" >> index.txt
echo "Enter Register Number to Search:"
read key
grep "^$key" index.txt
```

EXPERIMENT 14

File Allocation Strategies

C – Sequential Allocation

```
#include <stdio.h>

int main()
{
    int start,length,i;
    printf("Enter Starting Block: ");
    scanf("%d",&start);
    printf("Enter File Length (Number of Blocks): ");
    scanf("%d",&length);
    printf("\nAllocated Blocks:\n");
    for(i=0;i<length;i++)
        printf("%d ",start+i);
    printf("\n");
    return 0;
}
```

Shell – Sequential

```
#!/bin/bash
echo "Enter Starting Block:"
CS5302 Operating Systems – Xerox Code | Page
22read start
echo "Enter File Length:"
read length
echo "Allocated Blocks:"
for ((i=0;i<length;i++))
do
echo -n "$((start+i)) "
done
echo
```

C – Indexed Allocation

```
#include <stdio.h>
int main()
{
int n,indexBlock,blocks[20],i;
printf("Enter Index Block: ");
```

```
scanf("%d",&indexBlock);
printf("Enter Number of Blocks: ");
scanf("%d",&n);
printf("Enter Block Numbers:\n");
for(i=0;i<n;i++)
scanf("%d",&blocks[i]);
printf("\nIndex Block : %d\n",indexBlock);
printf("Allocated Blocks : ");
for(i=0;i<n;i++)
printf("%d ",blocks[i]);
printf("\n");
return 0;
}
```

Shell – Indexed

```
#!/bin/bash
```

```
echo "Enter Index Block:"
```

```
read index
```

```
echo "Enter Number of Blocks:"
```

```
read n
echo "Enter Block Numbers:"
for ((i=0;i<n;i++))
do
read block[$i]
done
echo "Index Block : $index"
echo -n "Allocated Blocks : "
for ((i=0;i<n;i++))
do
echo -n "${block[$i]} "
done
echo
```

C – Linked Allocation

```
#include <stdio.h>

int main()
{
int n,blocks[20],i;
```

```
printf("Enter Number of Blocks: ");
scanf("%d",&n);
printf("Enter Block Numbers:\n");
for(i=0;i<n;i++)
scanf("%d",&blocks[i]);
printf("\nLinked Allocation:\n");
for(i=0;i<n-1;i++)
printf("%d --> ",blocks[i]);
printf("%d --> NULL\n",blocks[n-1]);
return 0;
}
```

CS5302 Operating Systems – Xerox Code | Page

23Shell – Linked

```
#!/bin/bash
```

```
echo "Enter Number of Blocks:"
```

```
read n
```

```
echo "Enter Block Numbers:"
```

```
for ((i=0;i<n;i++))
```

```
do
read block[$i]
done
echo "Linked Allocation:"
for ((i=0;i<n-1;i++))
do
echo -n "${block[$i]} --> "
done
echo "${block[$((n-1))]} --> NULL"
```

EXPERIMENT 15

Disk Scheduling Algorithms

C – FCFS

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
int req[20],n,head,i;
int seek=0;
```

```
printf("Enter Number of Requests: ");
scanf("%d",&n);
printf("Enter Request Queue:\n");
for(i=0;i<n;i++)
scanf("%d",&req[i]);
printf("Enter Initial Head Position: ");
scanf("%d",&head);
for(i=0;i<n;i++)
{
seek+=abs(req[i]-head);
head=req[i];
}
printf("Total Head Movement = %d\n",seek);
return 0;
}
```

Shell – FCFS

```
#!/bin/bash
```

```
queue=(98 183 37 122 14 124 65 67)
```

```
head=53
seek=0
for req in "${queue[@]}"
do
diff=$((req-head))
if [ $diff -lt 0 ]
then
diff=$((-diff))
fi
seek=$((seek+diff))
head=$req
done
echo "Total Head Movement = $seek"
```

C – SSTF

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
```

```
int req[20],visited[20]={0};
```

```
int n,head,i,count=0;
```

```
int seek=0,index,min,distance;
```

CS5302 Operating Systems – Xerox Code | Page

```
24 printf("Enter Number of Requests: ");
```

```
scanf("%d",&n);
```

```
printf("Enter Request Queue:\n");
```

```
for(i=0;i<n;i++)
```

```
scanf("%d",&req[i]);
```

```
printf("Enter Initial Head Position: ");
```

```
scanf("%d",&head);
```

```
while(count<n)
```

```
{
```

```
min=9999;
```

```
for(i=0;i<n;i++)
```

```
{
```

```
if(!visited[i])
```

```
{
```

```
distance=abs(req[i]-head);
if(distance<min)
{
min=distance;
index=i;
}
}
}
seek+=min;
head=req[index];
visited[index]=1;
count++;
}
printf("Total Head Movement = %d\n",seek);
return 0;
}
```

Shell – SSTF

```
#!/bin/bash
```

```
echo "SSTF Disk Scheduling Demonstration"
echo "Request Queue : 98 183 37 122 14 124 65
67"
echo "Initial Head Position : 53"
echo "Total Head Movement calculated using
SSTF."
```

C – SCAN

```
#include<stdio.h>

int main()
{
int disk_size=200;
int head=53;
printf("SCAN Disk Scheduling\n");
printf("Initial Head Position : %d\n",head);
printf("Head moves towards higher cylinders,\n");
printf("then reverses direction.\n");
return 0;
}
```

Shell – SCAN

```
#!/bin/bash
```

```
echo "SCAN Disk Scheduling"
```

```
echo "Head moves in one direction and then  
reverses."
```

C – C-SCAN

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int head=53;
```

```
printf("C-SCAN Disk Scheduling\n");
```

```
printf("Initial Head Position : %d\n",head);
```

```
printf("Head moves in one direction.\n");
```

```
printf("After reaching the end, it returns to the  
beginning.\n");
```

```
return 0;
```

```
CS5302 Operating Systems – Xerox Code | Page  
25}
```

Shell – C-SCAN

```
#!/bin/bash
```

```
echo "C-SCAN Disk Scheduling"
```

```
echo "Head moves in one direction only."
```

```
echo "After reaching the end, it returns to the  
beginning."
```

CS5302 Operating Systems – Xerox Code | Page

26